

SCS1308
Foundations of Algorithm

Tutorial - 03

Solving recurrence equations

Methods of Solving recurrence equations

- Recursion tree method (Discussed Last Week) ✅
- Substitution method
- Iteration method
- Master Theorem

Last week Recap

1. Recursion Tree Method

Idea: *Draw the recursive calls as a tree and calculate the work at each level.*

```
void fun(int n) {

    if (n <= 1)
        return;

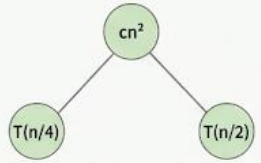
    fun(n / 4);
    fun(n / 2);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << "GFG ";
        }
    }
}
```

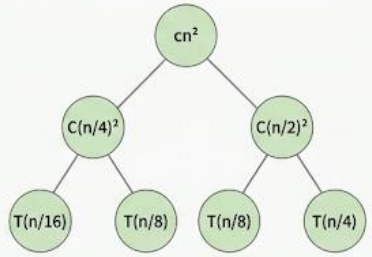
Recurrence relation is : $T(n) = T(n/4) + T(n/2) + O(n^2)$

01 Step | Consider the recurrence relation.

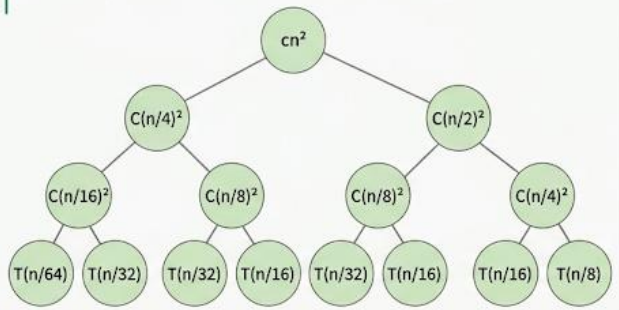
$$T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + cn^2$$



02 Step | If we further break down the expression T(n/4) and T(n/2), we get the following recursion tree.



03 Step | Breaking down further gives us following



The Pattern (Cost per Level)

- **Level 0:** cn^2
- **Level 1:** $\frac{5}{16}cn^2$
- **Level 2:** $\left(\frac{5}{16}\right)^2 cn^2$

Take the Sum (Decreasing GP)

$$T(n) \leq cn^2\left(1 + \frac{5}{16} + \left(\frac{5}{16}\right)^2 + \cdots\right)$$

Final Result: $T(n) = O(n^2)$

2. Substitution Method

Idea: *Guess the answer, then prove that your guess is correct using mathematical induction*.

Step 1: ***Guess a solution*** (use tree method)

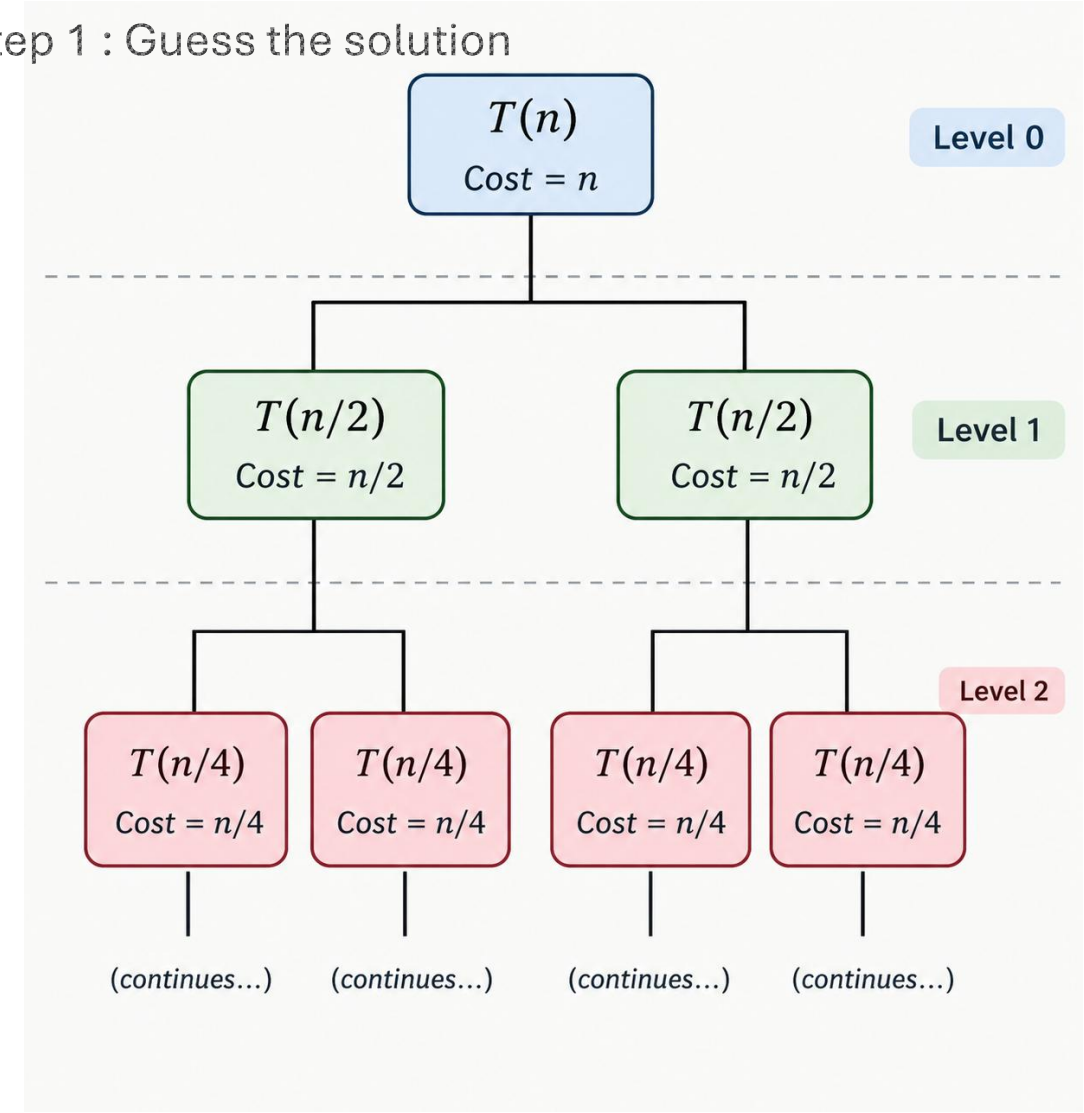
Step 2: ***Use Induction to Prove it***

- Use the recursion tree method to find a guess

$$\textbf{Example}: T(n) = \begin{cases} 1, & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + n, & if\ n > 1 \end{cases}$$

Example 1: $T(n) = \begin{cases} 1, & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + n, & if\ n > 1 \end{cases}$

Step 1 : Guess the solution



Level	Number of nodes	Cost per node	Total cost
0	1	n	n
1	2	$n/2$	n
2	4	$n/4$	n
3	8	$n/8$	n
...	...	...	n
$\log n$	n nodes	1	n

Number of Levels : $\log(n+1)$

Guess :

$$T(n) = n \cdot (\log(n) + 1) = n\log(n) + n$$

Continue→

Step 2 : Use Induction to Prove it

Example 1: $T(n) = \begin{cases} 1, & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + n, & if\ n > 1 \end{cases}$

Guess :

$$T(n) = n\log(n) + n$$

Base case:

for $n = 1$ ⟶ $1 \cdot \log(1) + 1 = 1 = T(n)$

Inductive steps:

$T(k) = k\ \log(k) + k$, for *all positive powers of two* $k < n$ ----- (1)

Use $k = n/2$ (which is less than n)

$$T(n) = 2T(n/2) + n$$

$$= 2\left[\frac{n}{2} \cdot \log\left(\frac{n}{2}\right) + \frac{n}{2}\right] + n$$

$$= n \cdot \log(\tfrac{n}{2}) + n + n$$

$$= n \cdot [\log(n) - \log(2)] + n + n$$ *(Don't forget $log(2) = log_2 2 = 1$)*

$$= n \cdot \log(n) + n$$ *(This is our guess)*

- By induction on powers of two, the formula holds for all $n = 2^k$, $k \geq 0$
- Conclusion. $T(n) = n\log(n) + n$.
- *Therefore*, $T(n) = \Theta(n \cdot \log n)$.

3. Iteration Method

Prerequires:
Sum of Geometric Progression
A/L math !

$$\sum_{i=0}^{n} a^n = \frac{a^{n+1} - 1}{a - 1}$$

Idea: *Expand the recurrence repeatedly until you see a pattern*.

1. **Expand the terms:** Repeatedly replace each term with its smaller subproblem.
2. **Identify the pattern:** Continue expanding until a clear general pattern emerges and you reach the base condition.
3. **Solve for the final value:** Plug in the known value of the base case to calculate the final expression.

$$\textbf{Example}: T(n) = \begin{cases} 1, & if\ n = 1 \\ 3T(n-1) + 1, & if\ n > 1 \end{cases}$$

$$T(n) = 3T(n-1) + 1 \quad \text{——— (1)}$$

Example : $$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ 3T(n-1) + 1, & \text{if } n > 1 \end{cases}$$

$$T(n-1) = 3T\big((n-1)-1\big) + 1 = 3T(n-2) + 1$$

$$T(n) = 3[3T[(n-2)+1] + 1 = 3^2T(n-2) + 3 + 1 \quad \text{——— (2)}$$

$$T(n-2) = 3T\big((n-2)-1\big) + 1 = 3T(n-3) + 1$$

$$T(n) = 3^2[3\,T(n-3) + 1] + 3 + 1 = 3^3\,T(n-3) + 3^2 + 3^1 + 1 \quad \text{——— (3)}$$

∘
∘
∘

$$T(n) = 3^kT(n-k) + \sum_{i=0}^{k-1} 3^i \quad \text{(General form)} \quad \text{——— (k)}$$

Important

The recurrence reaches the base case $T(1) = 1$ when $n - k = 1$, which gives $k = n - 1$.
Substituting $k = n - 1$ into the general form:

$$T(n) = 3^{n-1}T(1) + \sum_{i=0}^{n-2} 3^i$$

Continue→

$$T(n) = 3^{n-1}T(1) + \sum_{i=0}^{n-2} 3^i$$

Since $T(1) = 1$:

Example : $\boldsymbol{T(n) = \begin{cases} 1, & \boldsymbol{if\ n = 1} \\ 3T(n-1) + 1, & \boldsymbol{if\ n > 1} \end{cases}}$

Prerequires $\sum_{i=0}^{n} a^n = \frac{a^{n+1} - 1}{a - 1}$

$$T(n) = 3^{n-1} + \sum_{i=0}^{n-2} 3^i = \sum_{i=0}^{n-1} 3^i$$

Evaluate the geometric series for the closed-form solution:

$$T(n) = \sum_{i=0}^{n-1} 3^i = \frac{3^{(n-1)+1} - 1}{3 - 1} = \frac{3^n - 1}{2}$$

Final Answer:

$$T(n) = \frac{3^n - 1}{2} \cong \Theta(3^n)$$

3. Master Theorem

Idea: *Use a formula to solve certain divide-and-conquer recurrences without drawing a tree or expanding everything.*

- The **Master Theorem** is a theorem used to determine the asymptotic complexity of certain divide-and-conquer recurrence relations.

For a recurrence of the form:

$$T(n) = aT(n/b) + cn^k,$$
$$T(1) = d$$

$Where\ a \geq 1, \quad b \geq 2, \quad c > 0, \quad d \geq 0\ ,\ k \geq 0 \quad are\ constants.$

$$T(n) = \begin{cases} \Theta(n^k) & \text{if } a < b^k \\ \Theta(n^k \log n) & \text{if } a = b^k \\ \Theta(n^{\log_b a}) & \text{if } a > b^k \end{cases}$$

Case 1

Case 2

Case 3

$Case\ 1: a < b^k$

$$T(n) = 2T(n/2) + n^2$$

- $a = 2$
- $b = 2$
- $k = 2$
- $b^k = 4$
- $a < b^k \rightarrow 2 < 4$

$$\boxed{T(n) = \Theta(n^2)}$$

$Case\ 2: a = b^k$

$$T(n) = 2T(n/2) + n$$

- $a = 2$
- $b = 2$
- $k = 1$
- $b^k = 2$
- $a < b^k \rightarrow 2 = 2$

$$\boxed{T(n) = \Theta(n \log n)}$$

$Case\ 3: a > b^k$

$$T(n) = 4T(n/2) + n$$

- $a = 4$
- $b = 2$
- $k = 1$
- $b^k = 2$
- $a < b^k \rightarrow 4 > 2$

$$\boxed{T(n) = \Theta\left(n^{\log_2 4}\right) = \Theta(n^2)}$$

$$\boxed{T(n) = \Theta(n^2)}$$